EPICS V4 Normative Types

EPICS V4 Normative Types, Editors Draft, 16-Mar-2015

Editors:

Greg White, SLAC Bob Dalesio, BNL Mark Rivers, APS (Invited Expert) Marty Kraimer, BNL David Hickin, Diamond Light Source

Abstract

This document defines a set of standard high-level data types to aid interoperability of peers at the application level of an EPICS V4 network.

The abstract type definition and function of each such standard type is described. For instance, one such type defined here, named “NTTable”, defines a structure for expressing (in pvData) and communicating (using pvAccess) a table of numeric or string data.

The data types described here are approximately equivalent to EPICS V3 Database Request types (commonly known as “DBR” types), although Normative Types extend the concept to structured data and operate at a higher level in a complex control system, or data exchange, than DBR types. Also, Normative Types may be used purely for data exchange though the dynamic data exchange interfaces offered by EPICS pvAccess and pvData modules, such as pvDatabase or pvAccess RPC servers.

For more information about EPICS, please refer to the home page of the Experimental Physics and Industrial Control System.

Status of this Document

This revision of the Normative Types document is a minor modification to the 16 Mar 2015 version. This revision adds minor clarifications to the description of NTTable.

The 16 Mar 2015 version updates the definitions of time_t, control_t, display_t, and alarmLimit_t and changes the order of optional fields in a number of Normative Types. It replaces NTImage with NTNDArray, adds NTAttribute, NTMultiChannel, NTUnion and NTScalarMultiChannel and removes NTVariantArray and a number of types proposed in earlier drafts.

This version contains a number of types which use pvData unions.

It describes the new conventions for Normative Type IDs including versioning and namespaces. Type IDs for Normative Type structure fields are given.

The linguistic conventions used in the document have been overhauled.

See Appendix A for items that may be added to future revisions of this specification.

This version is an Editors Draft towards the First Public Working Draft. The First Public Working Draft will be intended for the EPICS community to review and comment. Resulting comments will drive subsequent revisions of the Normative Types specification and the EPICS V4 Working Group's reference implementations of software that helps create, populate and exchange Normative Type pvData.

Comments are welcome, though bear in mind this is a pre-public release version.

The terms MUST, MUST NOT, SHOULD, SHOULD NOT, REQUIRED, and MAY when highlighted (through style sheets, and in uppercase in the source) are used in accordance with RFC 2119 [RFC2119]. The term NOT REQUIRED (not defined in RFC 2119) indicates exemption.

Table of Contents

Introduction

The Normative Types described in this document are a set of software designs for high-level composite data types suitable for the application-level data exchange between EPICS network endpoints using the pvAccess protocol. In particular, they are intended for use in online scientific data services. The intention is that where the endpoints in an EPICS network use only Normative Types, each peer in the network should be able to understand all the data transmitted to it, at least syntactically, and be able to take processing steps appropriate to that data.

We call these types the Normative Types, to emphasize their role as the prescriptions of abstract data structures, whose role and intended semantics are described in this document, as opposed to implemented software; and that conformance to these semantics is a necessary condition for interoperability of using systems.

The EPICS (7) module pvData bib:pvdata supplies a typing mechanism and object management API for efficiently defining, creating, accessing and updating memory resident structured data. EPICS module pvAccess bib:pvaccess supports the efficient exchange of pvData defined data between EPICS V4 network peers. The EPICS V4 Normative Types specification defines some general purpose data types that build on pvData. These are designed to be generally applicable to the process control, and the software applications level, of scientific instruments.

A simple example of a Normative Type described in this document is the one for exchanging any single scalar value, such as one floating point number, one integer or one string. That Normative Type is named “NTScalar”. When a client receives a pvData datum which identifies itself as being of type NTScalar, the client will know to expect that the structure which carries the NTScalar will include the scalar value in question (along with its type), and that value may be accompanied by up to 5 additional fields: a description of the quality in question, a timestamp, an indication of alarm severity, fields that help in how to display the value, and data about its operating limits. See the example below.

An example of a simple Normative Type is the NTScalar:

```
NTScalar :=

structure
    scalar_t    value
    string      descriptor  :opt
    alarm_t     alarm       :opt
    time_t      timeStamp   :opt
    display_t   display     :opt
    control_t   control     :opt
```

A more complex example: If a client receives a pvData datum which identifies itself as being of type NTTable, this document specifies that it should expect the datum to contain 0 or more arrays of potentially different types. The description of NTTable in this document will say that the client should interpret the arrays as the columns of a table, and should render such a datum appropriately as a table, with row elements being taken from the same numbered elements of each array.

```
NTTable :=

structure
  string[]    labels           // The field names of each field in value
  structure   value
     {scalar_t[] colname}0+  // 0 or more scalar array type
                              // instances, the column values.
  string      descriptor    : opt
  alarm_t     alarm         : opt
  time_t      timeStamp     : opt
```

Description of Normative Types

All the EPICS V4 Normative Types are defined as particular structure instance definitions of a pvData structure. This is true even of the Normative Types describing simple values like a single int, since all Normative Types optionally include descriptor, alarm and timestamp. The fields of any given Ntype datum instance can be ascertained at runtime using the pvData Field introspection interface bib:pvdata.

See the Normative Type instance self-identification section below for more on how to examine a given pvData instance to see which fields it includes. That section also includes how to mark a pvData instance as a Normative Type, and how to look for that mark.

Definition: Normative Type

The Normative Types definitions in this document each have the following general form:

1. They are defined as structures, composed of fields.
2. They usually have one primary field called “value”, which encodes the most important data of the type.
3. They are composed of required fields, and optional fields. The required fields come first, the optional fields follow.
4. The order of fields matters. Although the Normative Types pvData binding allows for access though an introspection API, senders must encode the fields in the order described in this document.

Linguistic conventions used in this document

A Normative Type can be used both for sending data from client to service and from service to client. In this document we refer generally to an *agent*, being either a client or a server. If the agent is specifically at the user’s end, we call it the *user agent*. *Client* and *server* refer to the directionality of the transaction, server being the agent that is doing the sending.

The word “Ntype” is used as a short form of “Normative Type”.

The Normative Type data descriptions are given with the syntactic conventions and grammar given below. The types are described in a BNF-like syntax in order to add clear distinctions between symbol types, particularly terminality, recurrence, which names a user is expected to add and which are predefined. This syntax is essentially Extended Backus-Naur Form (EBNF), with some slight modifications to preserve the order of terms and the rules for line ends and indentation.

The syntactic conventions are as follows.

First, the conventions for terminal and non-terminal types are:

- *italics* - a non-terminal. These are used to stand for a choice of pvData type, or named sequence of fields, or for a specific structure or union, and hence non-terminal.

- `plaintext` - terminals. These will be either a pvData Meta Language keyword or a label. The Meta language keywords consist of `structure`, `union`, `any`, the scalar type keywords (`boolean`, `byte`, `short`, `int`, `long`, `double`, `ubyte`, `ushort`, `uint`, `ulong`, `float`, `double` and `string`) and the corresponding arrays `structure[]`, `union[]`, `any[]`, and scalar arrays (e.g. `int[]`, `double[]]`).
- <name> - A user-provided label name.A programmer using the Normative Type will choose what goes in the <>.

So, for example, scalar_t is non-terminal as it stands for a choice of pvData type and time_t is non-terminal because it stands for a particular structure. On the other hand, in the definition of *time_t*, `long` and `secondsPastEpoch` are a keyword and a label respectively, and so are terminal, and the columns of NTTable, <colname>, are user-provided labels.

In this section <> will also be used for describing patterns of definitions or meta rules such as production rules of the grammar to indicate a choice of terminal or non-terminal terms in the pattern or rule.

The EBNF-like syntax for definitions is used. A description consists of 3 terms - a left-hand side (LHS), a right-hand side (RHS), and the symbol ":=" separating them, which is to be interpreted as "LHS is defined as RHS". The LHS will be the non-terminal being defined. The RHS will be a sequence of terminal or non-terminal terms.

Note that in the definitions below line-ends (EOLs) are not explicitly specified. They are implied except when multiple lines are used to specify alternatives separated by |, where only the final EOL is implied.

The following EBNF symbols are also used:

- | - used to separate alternative items; one item is chosen from this list of alternatives.

- [] - optional items are enclosed between square brackets [and]; the item can either be included or discarded. Note, optional fields of structures are marked as such by the use of `:opt` instead of square brackets.
- { } - a sequence of occurrences of the item or items in the braces. The number of occurrences follows. 0+ means 0 or more. 1+ means 1 or more.

The following production rules are employed:

1. Replace a non-terminal by its definition, except where the non-terminal defines a structure or union and is followed by a field name. (The modified rule for non-terminal structures and unions is described below.)
2. Choose an alternative for items separated by |.
3. Choose a user supplied label for items between angle brackets (< and >).
4. Include or discard items between square brackets ([and]). Note this excludes a pair of square brackets ([]) used to signify an array.
5. Include or discard fields marked `:opt`.
6. For items between braces ({ and }) replace with an appropriate number of occurrences of the item. For a sequence of pvData fields a line-end (EOL) is implied after each one.

In the case of structure and union fields, to preserve the order of terms in the pvData Meta language, as well as obtaining appropriate indentation, the usual EBNF rule of replacing a non-terminal by its definition requires the following modification:

Suppose a non-terminal term has a definition of the form

```
<non-terminal>:=

structure
    fieldList
```

where:

<non-terminal>

The non-terminal term being defined.

<fieldName>

A choice of terminal or non-terminal terms describing a list of 0 or more pvData fields.

Then for a label (a field name), <fieldName>, the terms

```
<non-terminal> <fieldName>
```

are replaced by

```
structure <fieldName>
    <fieldList>
```

The result of the any substitution is suitably indented to preserve the logic of the pvData meta language.

Thus the structure derived from the definition of NTEnum below, with all optional fields present, is

```
structure
    structure  value
        int      index
        string[] choices
    string     descriptor
    structure  timeStamp
        long     secondsPastEpoch
        int      nanoseconds
        int      userTag
    structure   alarm
        int      severity
        int      status
        string   message
```

The same rule also applies with `union` in place of `structure`.

The grammar for a Normative Type definition follows the pattern below. That is, a Normative Type is defined as a structure composed of fields. A field may be optional, and may be described along with a comment:

```
<NormativeType>:=

structure
   { <pvDataField> [:opt] [// <commentText>] }1+
```

where:

<NormativeType>

The name of the Normative Type being defined.

<pvDataField>

A choice of terms defining a pvData field

:opt

Indicates that the preceding field is optional in the Normative Type.

// <commentText>

A field production element may be followed by a comment.

In most cases a Normative Type definition will be of the form

```
<NTname>:=

structure
   { ntfieldChoice fieldName [:opt] [// commentText] }1+
```

where:

<ntFieldChoice>

Terminal or non-terminal terms, possibly separated by |, from the valid Normative Type Fields as defined below.

<fieldName>

The identifier of the field. Usually a terminal label.

For example, a definition meeting this pattern would be

```
NTExample :=

structure
    enum_t | scalar_t   value
    int                 N                   // this field has a comment
    string              descriptor  :opt
    alarm_t             alarm       :opt
    time_t              timeStamp   :opt
```

Normative Type Fields

This section defines the fields that may appear in a Normative Type's definition.

Each field of a Normative Type will typically be one of the following:

```
ntfield :=

  scalar_t        // a simple numerical, boolean, or string value
| scalar_t[]      // an array of simple values
| enum_t          // an enumeration
| enum_t[]        // an array of enumerations
| time_t          // a point in time, used for timestamps
| time_t[]        // an array of points in time
| alarm_t         // a summary diagnostic of a control system event
| alarm_t[]       // an array of summary diagnostics
| alarmLimit_t    // value thresholds for a control system diagnostic report
| alarmLimit_t[]  // an array of threshold values
| display_t       // metadata of displayed data
| display_t[]     // an array of display metadata
| control_t       // control setpoint range boundaries
| control_t[]     // an array of control setpoint range boundaries
| any             // a variant union type
| any[]           // an array of variant unions fields
| ntunion_t       // a regular union storing ntfields only
| ntunion_t[]     // a regular union array storing ntfields only
| union_t         // any regular union
| union_t[]       // any regular union array
| anyunion_t      // any variant or regular union
| anyunion_t[]    // any variant or regular union array
```

although some examples may have fields of other types.

Simple Normative Type fields - scalar and scalar array types

Note that of all the Normative Type fields only *scalar_t* and *scalar_t[]* are of simple type, that is, having a single scalar or scalar array value of a fixed type. All the others are represented by a complex type, i.e. a structure or union or arrays of structures or unions (see Structured Normative Type fields and Union Normative Type fields below).

scalar_t

The field is a scalar value. Scalar fields would be implemented with pvData field Type “scalar”:

```
scalar_t :=

   boolean  // true or false
|  byte     // 8 bit signed integer
|  ubyte    // 8 bit unsigned integer
|  short    // 16 bit signed integer
|  ushort   // 16 bit unsigned integer
|  int      // 32 bit signed integer
|  uint     // 32 bit unsigned integer
|  long     // 64 bit signed integer
|  ulong    // 64 bit unsigned integer
|  float    // single precision IEEE 754
|  double   // double precision IEEE 754
|  string   // UTF-8 *
```

scalar_t[]

The field is an array of scalars. Scalar array fields would be implemented with a pvData field of type “scalarArray”:

```
scalar_t[] :=

   boolean[]  // array of true or false
|  byte[]     // array of 8 bit signed integer
|  ubyte[]    // array of 8 bit unsigned integer
|  short[]    // array of 16 bit signed integer
|  ushort[]   // array of 16 bit unsigned integer
|  int[]      // array of 32 bit signed integer
|  uint[]     // array of 32 bit unsigned integer
|  long[]     // array of 64 bit signed integer
|  ulong[]    // array of 64 bit unsigned integer
|  float[]    // array of single precision IEEE 754
|  double[]   // array of double precision IEEE 754
|  string[]   // array of UTF-8 *
```

Structured Normative Type fields

This subsection defines those fields of a Normative Type structure definition that are themselves structures or arrays of structures.

The structured Normative Type fields would be implemented with type pvData field type "structure" or "structureArray".

enum_t

An *enum_t* describes an enumeration. The field is a structure describing a value drawn from a given set of valid values also given. It is implemented as a pvData Field of type "structure" of type ID "enum_t" with the following form:

```
enum_t :=

structure
    int index
    string[] choices
```

where:

index

The index of the current value of the enumeration in the array choices below.

choices

An array of strings specifying the set of labels for the valid values of the enumeration.

enum_t[]

An *enum_t[]* describes an array of enumerations. The field is an array of structures each describing a value drawn from a given set of valid values also given in each. It is implemented as a pvData field of type "structureArray", each element of which is a structure of the form *enum_t* above.

time_t

A *time_t]* describes a defined point in time. The field is a structure describing a time relative to midnight on January 1st, 1970 UTC. It is implemented as a pvData field of type "structure" of type ID "time_t" and with the following form:

```
time_t :=

structure
    long secondsPastEpoch
    int  nanoseconds
    int  userTag
```

where:

secondsPastEpoch

Seconds since Jan 1, 1970 00:00:00 UTC.

nanoseconds

Nanoseconds relative to the `secondsPastEpoch` field.

userTag

An integer value whose interpretation is deliberately undefined and therefore MAY be used by EPICS V4 agents in a user defined way.

Interpretation: The point in time being identified by a *time_t*, is given by Jan 1, 1970 00:00:00 UTC plus some nanoseconds given by its `secondsPastEpoch` times 10^9 plus its `nanoseconds`.

time_t[]

A *time_t[]* describes an array of points in time. The field is an array of structures each describing a time relative to January 1st, 1970 UTC. It is implemented as a pvData field of type "structureArray", each element of which is a structure of the form *time_t* above.

alarm_t

An *alarm_t* describes a diagnostic of the value of a control system process variable. It indicates essentially whether the associated value is good or bad, and whether agent systems should alert people to the status of the process.

Processes in EPICS V3 and V4 IOCs include extensive support for evaluating alarm conditions. The definition of the fields in an `alarm` are given in bib:epicsrecref. The field is a structure describing an alarm. It is implemented as a pvData field of type "structure" of type ID "alarm_t" with the following form:

```
alarm_t :=

structure
    int severity
    int status
    string message
```

where:

severity

severity is defined as an int (not an *enum_t*), but MUST be functionally interpreted as the enumeration {noAlarm, minorAlarm, majorAlarm, invalidAlarm, undefinedAlarm } indexed from noAlarm=0 bib:epicsrecref.

status

status is defined as an int (not an *enum_t*), but MUST be functionally interpreted as the enumeration {noStatus, deviceStatus, driverStatus, recordStatus, dbStatus, confStatus, undefinedStatus, clientStatus } indexed from noStatus=0 bib:epicsrecref.

message

A message string.

Interpretation MUST be as with V3 IOC record processing, as described in the EPICS Reference Manual bib:epicsrecref.

alarm_t[]

An *alarm_t[]* is an array of alarm conditions. The field is an array of structures each describing an alarm condition. It is implemented as a pvData field of type "structureArray", each element of which is a structure of the form *alarm_t* above.

alarmLimit_t

An *alarmLimit_t* is a structure that gives the numeric intervals to be used for the high and low limit ranges of an associated value field. The specific value to which the alarmLimit refers, is not specified in the alarmLimit structure. It is usually a value field of type double that appears in the same

structure as the alarmLimit. *alarmLimit_t* is implemented as a pvData field of type "structure" of type ID "alarmLimit_t" with the following form:

```
alarmLimit_t :=

structure
    boolean active
    double lowAlarmLimit
    double lowWarningLimit
    double highWarningLimit
    double highAlarmLimit
    int lowAlarmSeverity
    int lowWarningSeverity
    int highWarningSeverity
    int highAlarmSeverity
    double hysteresis
```

where:

active

Is alarming active? If no then alarms are not raised. If yes then the associated value is checked for alarm conditions.

lowAlarmLimit

If the value is <= lowAlarmLimit then the severity is lowAlarmSeverity.

lowWarningLimit

If the value is > lowAlarmLimit and <= lowWarningLimit then the severity is lowWarningSeverity.

highWarningLimit

If the value is >= highWarningLimit and < highAlarmLimit then the severity is highWarningLimit.

highAlarmLimit

If the value is >= highAlarmLimit then the severity is highAlarmSeverity.

lowAlarmSeverity

Severity for value that satisfies lowAlarmLimit.

lowWarningSeverity

Severity for value that satisfies lowWarningLimit.

highWarningSeverity

Severity for value that satisfies highWarningLimit.

highAlarmSeverity

Severity for value that satisfies highAlarmLimit.

hysteresis

When a value enters an alarm limit this is how much it must change before is it put into a lower severity state. This prevents alarm chatter.

Code that checks for alarms should use code similar to the following:

```
boolean active = pvActive.get();
if(!active) return;
double  val = pvValue.get();
int severity = pvHighAlarmSeverity.get();
double level = pvHighAlarmLimit.get();
if(severity>0 && (val>=level)) {
    raiseAlarm(level,val,severity,"highAlarm");
    return;
}
severity = pvLowAlarmSeverity.get();
level = pvLowAlarmLimit.get();
if(severity>0 && (val<=level)) {
    raiseAlarm(level,val,severity,"lowAlarm");
    return;
}
severity = pvHighWarningSeverity.get();
level = pvHighWarningLimit.get();
if(severity>0 && (val>=level)) {
    raiseAlarm(level,val,severity,"highWarning");
    return;
}
severity = pvLowWarningSeverity.get();
level = pvLowWarningLimit.get();
if(severity>0 && (val<=level)) {
    raiseAlarm(level,val,severity,"lowWarning");
    return;
}
raiseAlarm(0,val,0,"");
```

NOTE: The current pvData implementations have a structure named **valueAlarm_t** instead of **alarmLimit_t**. *valueAlarm_t* is similar to *alarmLimit_t*, except that the former's alarm limit fields (`lowAlarmLimit`, `lowWarningLimit`, `highWarningLimit` and `highAlarmLimit`) can be any integer or floating point scalar type (the same type for all the limit fields in each case), rather than only double. There is also a separate form for alarm limits for boolean values. *alarmLimit_t* is identical to the *valueAlarm_t* for type double, except that the type ID of *valueAlarm_t* is "valueAlarm_t"). Normative types only defines alarmLimit since this is what clients like plot tools use.

alarmLimit_t[]

An *alarmLimit_t[]* is an array of alarm limit conditions. The field is an array of structures each describing an alarm limit. It is implemented as a pvData field of type "structureArray", each element of which is a structure of the form *alarmLimit_t* above.

display_t

A *display_t* is a structure that describes some typical attributes of a numerical value that are of interest when displaying the value on a computer screen or similar medium. The `units` field SHOULD contain a string representation of the physical units for the value, if any. The `description` field SHOULD contain a short (one-line) description of what the value represents, such as can be used as a label in a display. The fields `limitLow` and `limitHigh` represent the range in between which the value should be presented as adjustable.

The field is a structure describing a *display_t*. It is implemented as a pvData field of type "structure" of type ID "display_t" with the following form:

```
display_t :=

structure
    double limitLow
    double limitHigh
    string description
    string units
    int precision
    enum_t form(3)
        int index
        string[] choices ["Default", "String", "Binary", "Decimal", "Hex", "Exponential",
"Engineering"]
```

where:

limitLow

The lower bound of range within which the value must be set, to be presented to a user.

limitHigh

The upper bound of range within which the value must be set, to be presented to a user.

description

A textual summary of the variable that the value quantifies.

precision

Number of decimal points that are displayed when formatting a floating point number. This corresponds to the PREC field in EPICS database records with floating point values (e.g., ai, ao, calc, calcout record types.)

form

An enumeration to specify formatting a value to be displayed. By default, a floating point number is formatted with the number of decimal points defined in the precision field. Formatting of an EPICS database record value can be configured by including eg. info(Q:form, “Hex”) in record definition.

units

The units for the value field.

Where an *display_t* structure instance is present in a Normative Type structure, it MUST be interpreted as referring to that Normative Type’s field named “value”. Therefore it is only used in Normative Types that have a single numeric “value” field.

display_t[]

A *display_t[]* is an array of *display_t*. The field is an array of structures each describing the display media oriented metadata of some corresponding process variable value, as described by *display_t* above. It is implemented as a pvData field of type “structureArray”, each element of which is a structure of the form *display_t* above.

control_t

A *control_t* is a structure that describes a range, given by the interval (limitLow,limitHigh), within which it is expected some control software or hardware shall bind the control PV to which this Normative Type instance's value field refers as well as a minimum step change of the control PV.

The field is a structure describing a *control_t*. It is implemented as a pvData field of type "structure" of type ID "control_t" with the following form:

```
control_t :=

structure
    double limitLow
    double limitHigh
    double minStep
```

where:

lowLimit

The control low limit for the value field.

highLimit

The control high limit for the value field.

minStep

The minimum step change for the value field.

control_t[]

A *control_t[]* is an array of *control_t*. The field is an array of structures each describing the setpoint range interval of some process variable. It is implemented as a pvData field of type "structureArray", each element of which is a structure of the form *control_t* above.

Union Normative Type fields

This subsection defines those fields of a Normative Type structure definition that are unions or arrays of unions.

The union NormativeType fields are implemented with pvData fields of type “union” or “unionArray”.

The union Normative Type fields consist of the variant union `any` and variant union array `any\[\]` as well as a number of non-terminal terms:

any

This is a field which is a variant union and is implemented using the pvData field type “union”.

any[]

This is a field that is an array of `any`, implemented using the pvData field type “unionArray”.

ntunion_t

ntunion_t stands for any regular union of ntfields and is implemented using the pvData field type “union”:

```
ntunion_t :=

union
    {ntfield  field-name}1+  // 1 or more ntfields.
```

ntunion_t[]

An *ntunion_t[]* stands for an array of unions, where the union is any regular union of 1 or more ntfields. It is implemented as a pvData field of type "unionArray" each element of which is a union (the same one in each case) of the form *ntunion_t* above.

union_t

union_t stands for any regular union of pvData fields and is implemented using the pvData field of type "union":

```
union_t :=

union
    {pvDataField}1+ // 1 or more pvData fields.
```

where:

pvDataField

Stands for any pvData field.

union_t[]

A *union_t[]* stands for an array of unions, where the union is any regular union of 1 or more pvData fields. It is implemented as a pvData field of type "unionArray" each element of which is a union (the same one in each case) of the form *union_t* above.

anyunion_t

anyunion_t stands for a variant union or any regular union of pvData fields and is implemented using the pvData field type "union":

```
anyunion_t:=

any | union_t
```

anyunion_t[]

An *anyunion_t[]* stands for a variant union array or a regular union array of any type an array of unions, where the union is any regular union of 1 or more pvData fields. It is implemented as a pvData field of type "unionArray" each element of which is a union (the same one in each case) of the form *anyunion_t* above:

```
anyunion_t[]:=

any[] | union_t[]
```

Normative Type Metadata

Metadata are included in runtime instances of Normative Types. The metadata includes to which Normative Type the structure instance conforms, version information, and other data to aid efficient processing, diagnostics and displays.

Normative Type instance self-identification

Normative Type instance data MUST identify themselves as such by including an identifying string. That is the Normative Type Identifier, or "Ntype Identifier" string for short. In the pvData binding of Normative Types, this string is carried in the type ID, added automatically to every pvData structure.

A Normative Type Identifier MUST be considered to be "case sensitive."

The namespace Name of EPICS Normative Types (which is used as the prefix for their pvData type ID), is the following:

```
epics:nt
```

The normative list of the Normative Type Identifiers corresponding to this draft of the EPICS V4 Normative Types specification document (this document), is given in Appendix B

As an example, one of the simplest Normative Types is NTScalar. It has formal Type Name “NTScalar”. Therefore, the Normative Type Identifier for an NTScalar, is presently epics:nt/NTScalar:1.0.

At present it is envisaged that the same namespace value shall be used for all versions of this document prior to Recommendation, including all Public Working Drafts of this document and those marked Last Call or similar.

pvAccess binding type identification

In the EPICS v4 pvData/pvAccess binding, the structure identification string (ID) of pvData structures is used to communicate the Normative Type of the datum carried by the pvData structure. Every pvData datum which is intended to conform to a Normative Type, MUST identify the Normative Type to which it conforms through its type ID. Its ID MUST have the value of its Normative Type Identifier. For instance, a pvData structure conforming to NTScalar, must have ID equal to “epics:nt/NTScalar:1.0”. Every EPICS V4 agent which is encoding or decoding pvData data that is described by Normative Types, SHOULD examine the ID of such data, to establish the Normative Type to which each datum conforms.

Example pvAccess/pvData binding

Recall that in the pvData system, data variables are constructed in two equally important parts; the introspection interface, in which data types are defined, and the data interface, in which instance variables are created and populated. The introspection interface can be used to examine an existing instance, to see what fields it possesses. Getting and setting values, is done through the data interface. As a programmer, you have to define both parts, the introspection interface of your type, and its data interface. Both the data and the introspection interfaces are exchanged by pvAccess. That is, when a sender constructs a data type, such as one conforming to an Normative Type, plus an instance of that type, and it sends the instance to a receiver, the receiver can check that the instance indeed contains the member fields it should find for that type, using the type's introspection interface.

The following Java code snippets give an example of the use of a pvData structure of Normative Type NTScalar, as defined below. in this example we show code as may be included in a trivial "multiplier" service, and a client of the multiplier service.

Sender

The sender typically first creates an introspection definition, using the pvData introspection interfaces (Field, Structure etc.). It then creates an instance of the type and populates it with the pvData data interfaces (PVField, PVStructure etc.).

Example of creating the introspection interface of an NTScalar, as may be done on a server that will be returning one. In this example, only one of the optional fields of NTScalar, named "descriptor" is included, along with the required field named "value".

```
// Create the data type definition, using the pvData introspection interface (Structure etc.).
FieldCreate fieldCreate = FieldFactory.getFieldCreate();
Structure resultStructure = fieldCreate.createStructure( "epics:nt/NTScalar:1.0",
    new String[] { "value", "descriptor" },
    new Field[] { fieldCreate.createScalar(ScalarType.pvDouble),
                  fieldCreate.createScalar(ScalarType.pvString) } );
```

Subsequently, the sender would create an instance of the type, and populate it.

Example of creating an instance and data interface of an NTScalar, as may be done on a data server, and populating it.

```
// If a and b were arguments to this service, the following creates an instance of
// a resultStructure, which conforms to the NTScalar Normative Type definition,
// and populates it. It would then return this PVStructure instance.
PVStructure result = PVDataFactory.getPVDataCreate().createPVStructure(resultStructure);
result.getDoubleField("value").put(a * b);
result.getStringField("descriptor").put("The product of arguments a and b");
```

The PVStructure instance, in the example called “result” would be returned to the receiver.

Receiver

Having in some way done a pvAccess get, the receiver could simply extract the primary value:

```
PVStructure result = easyPVA.createChannel("multiplierService").createRPC().request(request);
double product = result.getDoubleField("value").get();
```

A well written receiver would check that the introspection interface (Structure etc.) says that the received instance is indeed of the type it expects. It may extract the data fields individually, checking their type. Importantly, it can also see which optional fields it received, before attempting to access them. Here is a more complete receiver example for the NTScalar sent above. This code might be in the client side of the Multiplier service.

Example of a receiver of an NTScalar. The example checks that the returned pvData datum was an instance of an NTScalar, extracts the required value field, and then, if it’s present, extracts the optional “descriptor” field.

```
// Call the multiplier service sending the request in a structure
PVStructure result = easyPVA.createChannel("multiplierService").createRPC().request(request);

// Examine the returned structure via its introspection interface, to check whether its
// identifier says that it is a Normative Type, and the type we expected.
if (!result.getStructure().getID().equals("epics:nt/NTScalar:1.0"))
{
    System.err.println("Unexpected data identifier returned from multiplierService: " +
       "Expected Normative Type ID epics:nt/NTScalar:1.0, but got "
       + result.getStructure().getID());
    System.exit(-1);
}

// Get and print the required value member field as a Double.
System.out.println( "value = " + result.getDoubleField("value").get());

// See if there was also the descriptor subField, and if so, get it and print it.
PVString descriptorpv = (PVString)result.getSubField("descriptor");
if ( descriptorpv != null)
   System.out.println( "descriptor = " + descriptorpv.get());

// Or just print everything we got:
System.out.println("\nWhole result structure toString =\n" + result);
```

Future of type identification

In future drafts of this specification, a pattern to create extensions to the EPICS V4 Normative Types may be presented. It may be based on a formalized link to the XML namespace and XML Schema system, whereby the namespace part of the Normative Type Identifier of a datum whose type is an extension of one of these Normative Types, is replaced by another namespace that extends this one through an XML Schema out of band. In that case, the type name part would identify a type in that other namespace, though it may extend a type in this namespace.

Standard optional metadata fields

All of the Normative Types defined below, optionally include a descriptor, alarm and timestamp. There is no required interpretation of these fields, and therefore their meaning is not further described in the Normative Type definitions. Additionally, Normative Types may have other

optional fields, as defined individually below.

Optional descriptor field

An object of Normative Type may optionally include a field named “descriptor” and of type string, to be used to give identity, name, or sense information. For instance, it may be valued with the name of a device associated with control data, or the run number of a table of model data.

```
string descriptor  :opt     // Contextual information
```

Optional alarm field

An object of Normative Type may optionally include an alarm field.

```
alarm_t alarm      :opt     // Control system event summary
```

Optional timeStamp field

An object of Normative Type may optionally include a timeStamp field.

```
time_t timeStamp   :opt     // Event time
```

General Normative Types

The General Normative Types are for encapsulating data of any kind of application or use case. Compare to Specific Normative Types, defined later in this document, which are oriented to particular use cases.

NTScalar

NTScalar is the EPICS V4 Normative Type that describes a single scalar value plus metadata:

```
NTScalar :=

structure
    scalar_t    value
    string      descriptor  :opt
    alarm_t     alarm       :opt
    time_t      timeStamp   :opt
    display_t   display     :opt
    control_t   control     :opt
```

where:

value

The primary data carried by the NTScalar object. The field must be named “value” and can be of any simple scalar type as defined above.

NTScalarArray

NTScalarArray is the EPICS V4 Normative Type that describes an array of values, plus metadata. All the elements of the array of the same scalar type.

```
NTScalarArray :=

structure
    scalar_t[]  value
    string      descriptor  :opt
    alarm_t     alarm       :opt
    time_t      timeStamp   :opt
    display_t   display     :opt
    control_t   control     :opt
```

where:

value

The primary data carried by the NTScalarArray object. The field must be named “value” and can be of any scalar array type as defined above.

NTEnum

NTEnum is an EPICS V4 Normative Type that describes an enumeration (a closed set of possible values each described by an n-tuple).

```
NTEnum :=

structure
    enum_t      value
    string      descriptor  :opt
    alarm_t     alarm       :opt
    time_t      timeStamp   :opt
```

where:

value

The primary data carried by the NTEnum object. The field must be named “value” and must be an enumeration as defined above.

NTMatrix

NTMatrix is an EPICS V4 Normative Type used to define a matrix, specifically a 2-dimensional array of real numbers.

```
NTMatrix :=

structure
    double[]    value
    int[2]      dim         :opt
    string      descriptor  :opt
    alarm_t     alarm       :opt
    time_t      timeStamp   :opt
    display_t   display     :opt
```

where:

value

The numerical data comprising the matrix. The value is given as a single array of doubles. When `value` holds the data of a matrix, rather than a vector, then the data MUST be laid out in “row major order”; that is, all the elements of the first row, then all the elements of the second row, and so on. For instance, where NTMatrix represented a 6x6 matrix, element (1,2) of the matrix would be in the 2nd element of `value`, and element (3,4) would be in the 16th element.

dim

`dim` indicates the dimensions of the matrix. If `dim` is not present, `value` MUST be interpreted as a vector, of length equal to the number of elements of `value`. If `dim` is present, then it must have 1 or 2 elements; its one element value or both elements values MUST be > 0, and the number of elements in `value` MUST be equal to the product of the elements of `dim`. If `dim` is

present and contains a single element, then the NTMatrix MUST be interpreted as describing a vector. A `dim]` of 2 elements describes a matrix, where the first element of `dim` gives the number of rows, and the second element of `dim` gives the number columns. If `dim` is present and contains 2 elements, of which the first is unity, and the second is not (therefore is >1) then the NTMatrix MUST be interpreted as describing a row vector. If `dim` is present as contains 2 elements, of which the second is unity, and the first is not (therefore is >1) then the NTMatrix MUST be interpreted as describing a column vector.

User agents that print or otherwise render an NTMatrix SHOULD print row vector, column vector, and non-vector matrices appropriately.

NTURI

NTURI is the EPICS V4 Normative Type that describes a Uniform Resource Identifier (URI) bib:uri. Specifically, NTURI carries the four parts of a “Generic URI”, as described in bib:uri as the subset of URI that share a common syntax for representing hierarchical relationships within the namespace. As such, NTURI is intended to be able to encode any generic URI scheme’s data. However, NTURI’s primary purpose in the context of EPICS, is to offer a well formed and standard compliant way that EPICS agents can make a request for an identified resource from a channel, especially an EPICS V4 RPC channel. See ChannelRPC.

The “pva” scheme is introduced here for EPICS V4 interactions. The pva scheme implies but does not require use of the pvAccess protocol. A scheme description for Channel Access (implying the ca protocol) will be added later. What follows is a description of the syntax and semantics for the pva scheme.

```
NTURI  :=

structure
    string scheme
    string authority    : opt
    string path
    structure query     : opt
        {string | double | int <field-name>}0+
    {<field-type> <field-name>}0+
```

Interpretation of NTURI under the “pva” scheme

The following describes how the fields of the NTURI must be interpreted when the scheme is “pva”:

scheme

The scheme name must be given. For the pva scheme, the scheme name is “pva”. The pva scheme implies but does not require use of the pvAccess protocol.

authority

If given, then the IP name or address of an EPICS network pvAccess or channel access server.

path

The path gives the channel from which data is being requested.

query

A name value system for passing parameters. The types of the argument value MUST be drawn from the following restricted set of scalar types: double, int, or string.

<field-type>

Zero or more pvData Fields whose type are not defined until runtime, may be added to an NTURI by an agent creating an NTURI. This is the mechanism by which complex data may be sent to a channel. For instance a table of magnet setpoints.

The channel name given in the path MAY BE the name of an RPC channel. In that case, it's important to note that this specification makes no normative statement about where in the NTURI is encoded the name of the entity *about which* the RPC service is being called. For instance, an archive service, that gives the historical values of channels, may advertise itself as being on a single channel called say "archive service" (so the NTURI path field in that case would be set to "archiveservice", and in that case, the name of the EPICS channel about which archive data is wanted might well be encoded into one of the NTURI's query field parameters. Alternatively, the archive service might advertise a number of channels, each named perhaps after the channels whose historical data is being requested. For instance, a path may be "quad45:bdes;history", if that was the name of one of the channels offered by the archive service. An example of this second form is given below.

Use of NTURI may be explained by example. The following is an example client side of Channel RPC exchange, where a notional archive service, is asked for the data for a PV between two points in time. In this example, the archive service is advertising the channel name "quad45:bdes;history". Presumably, that service knows the archive history of a (second) channel, named probably, "quad45:bdes".

Construct the introspection interface (i.e. type definition) of the NTURI conformant structure that will be used to make requests to the archive service.

```
// Construct an NTURI for making a request to a service that understands
// query arguments named "starttime" and "endtime".
FieldCreate fieldCreate = FieldFactory.getFieldCreate();
Structure queryStructure = fieldCreate.createStructure(
    new String[] {"starttime", "endtime"},
    new Field[] { fieldCreate.createScalar(ScalarType.pvString),
                  fieldCreate.createScalar(ScalarType.pvString)});
Structure uriStructure =
    fieldCreate.createStructure("epics:nt/NTURI:1.0",
         new String[] { "path", "query" },
         new Field[] { fieldCreate.createScalar(ScalarType.pvString),
                       queryStructure } );
```

Populate our uriStructure (conformant to NTURI) with a specific request.

```
// Get a EasyPVA singleton.
EasyPVA easyPVA = EasyPVAFactory.get();

// Construct an NTURI with which to ask for the archive data of quad45:bdes
PVStructure request = PVDataFactory.getPVDataCreate().
        createPVStructure(uriStructure);
request.getStringField("path").put("quad45:bdes;history");
PVStructure query = request.getStructureField("query");
query.getStringField("starttime").put("2011-09-16T02.12.55");
query.getStringField("endtime").put("2011-09-16T10.01.03");

// Ask for the data, using the NTURI
PVStructure result =
easyPVA.createChannel(request.getStringField("path").get()).createRPC().request(request);
if ( result != null )
    System.out.println("The URI request structure:\n" + request
                +"\n\nResulted in:\n" + result);
```

The server side is not illustrated, but clearly its code would have registered a number of ChannelRPC services, each named after the PV whose historical data it offered.

NTNameValue

NTNameValue is the EPICS V4 Normative Type that describes a system of name and scalar values.

Use cases: In a school, a single NTNamedValue might describe the grades from a number of classes for one student.

```
NTNameValue :=

structure
    string[]     name
    scalar_t[]   value
    string       descriptor   :opt
    alarm_t      alarm        :opt
    time_t       timeStamp    :opt
```

where:

name

The keys associated with the 'value `field. Each element of` name `identifies the same indexed element of the` value` field, using a string label.

value

The data values, each element of which is associated with the correspondingly indexed element of the `name` field.

Each name (or "key") in the array of names, MUST be interpreted as being associated with its same indexed element of the `value` array.

NTTable

NTTable is the EPICS V4 Normative Type suitable for column-oriented tabular datasets.

An NTTable is made up of a number of arrays. Each array can be thought of as a column. Each array MUST be of a scalar type and all the arrays MUST be of the same length. Each array may be of a different scalar type. The set of the *i*th array members of all the columns make up one row, or n-tuple. The number of elements of `labels` MUST be equal to the number of fields of `value`.

Use case examples: a table of the Twiss parameters of all the lattice elements in an accelerator section. Another example, where the columns might vary call-to-call to an RPC setting, would be that of an EPICS V4 SQL database service. In that example one NTTable returned by the service would contain the tabular results of a SQL SELECT, essentially a recoded JDBC or ODBC ResultSet - see the rdbservice.

```
NTTable :=

structure
    string[]   labels             // Very short text describing each field below, i.e. column labels
    structure  value
        {scalar_t[]  colname}0+ // 0 or more scalar array instances, the column values.
    string     descriptor  : opt
    alarm_t    alarm       : opt
    time_t     timeStamp   : opt
```

where:

labels

The table column headings are given by the `labels` field. Each column heading given as one element of the array of strings.

value

The data of the table are encoded in a structure named `value`. The columnar data field is named “value” (rather than, for instance, “columndata”) so that the primary field of the type is named the same for all Normative Types. That helps general purpose clients identify the primary field of any Normative Type instance.

Interpretation

An NTTable instance represents a table of data. The column data is given in scalar arrays in the structure field `value`, and the column headings are given in field `labels`. Each / scalar array field of `value` contains the data for the column corresponding to the same indexed element of the `labels` field. Agents SHOULD use the elements of `labels` as the column headings. *There is no normative requirement that the field names of ``value`` match the strings in ``labels``.*

Note that the above description is given in terms of a table and its columns, but there is nothing specifically columnar about how this data may be rendered. A user may choose to print the fields row wise if, for instance, if there are many fields in `value`, but each has only length 1 or 2. For example, if one wanted to give all the scalar data related to one device, then one might use an NTTable rendered in such a way.

Validation

The number of *scalar_t[]* fields in the value structure, and the length of `labels` MUST be the same. All *scalar_t[]* fields in the `value` structure MUST have the same length, which is the number of “rows” in the table.

NTAttribute

NTAttribute is the EPICS V4 Normative Type for a named attribute of any type. It is is essentially a key-value pair which optionally can be tagged with additional strings.

This allows, for example, a collection of attributes to be queried on the basis of attribute name or tags.

```
NTAttribute :=

structure
    string    name
    any       value
    string[]  tags          : opt
    string    descriptor    : opt
    alarm_t   alarm         : opt
    time_t    timeStamp     : opt
```

where:

name

The name of the attribute. The “key” of the key-value pair.

value

The value of the attribute. The “value” of a key-value pair.

tags

Additional tags that an attribute can carry.

Specific Normative Types

The “Specific Normative Types” below are types oriented towards application-level scientific and engineering use cases. Compare to General Normative Types defined above. The currently defined types are each described in a section below.

Unless otherwise stated:

- Times MUST be in seconds
- Frequencies MUST be in Hz.

NTMultiChannel

NTMultiChannel is an EPICS V4 Normative Type that aggregates an array of values from different EPICS Process Variable (PV) channel sources, not necessarily of the same type, into a single variable.

```
NTMultiChannel :=

structure
    anyunion_t[]  value              // The channel values
    string[]      channelName        // The channel names
    string        descriptor         :opt
    alarm_t       alarm              :opt
    time_t        timeStamp          :opt
    int[]         severity           :opt
    int[]         status             :opt
    string[]      message            :opt
    long[]        secondsPastEpoch   :opt
    int[]         nanoseconds        :opt
    int[]         userTag            :opt
```

where:

value

The value from each channel.

channelName

The name of each channel.

alarm

The alarm associated with the NTMultiChannel itself. `severity`, `status`, and `message` show the alarm for each channel.

timeStamp

The timestamp associated with the NTMultiChannel itself. `secondsPastEpoch`, `nanoseconds` and `userTag` show the timestamp for each channel.

severity

The alarm severity associated with each channel.

status

The alarm status associated with each channel.

message

The alarm message associated with each channel.

secondsPastEpoch

The `secondsPastEpoch` field of the timestamp associated with each channel.

nanoseconds

The `nanoseconds` field of the timestamp associated with each channel.

userTag

The `userTag` field of the timestamp associated with each channel.

NTNDArray

NTNDArray is an EPICS Version 4 Normative Type designed to encode data from detectors and cameras, especially areaDetector applications. The type is heavily modeled on areaDetector's NDArray class. One NTNDArray gives one frame.

The definition of NTNDArray in full is:

```
NTNDArray :=

structure
    value_t       value
    codec_t       codec
    long          compressedSize
    long          uncompressedSize
    dimension_t[] dimension
    int           uniqueId
    time_t        dataTimeStamp
    NTAttribute[] attribute
    string        descriptor  :opt
    alarm_t       alarm       :opt
    time_t        timeStamp   :opt
    display_t     display     :opt
```

The meaning of the above fields, the definition of *value_t* and of *dimension_t* and the additional requirements for NDAttribute are described below. To simplify this the NTNDArray can be regarded as being composed of the following parts:

```
NTNDArray :=

structure
    Image data and codec
    Data sizes
    Dimensions
    Unique ID and data timestamp
    Attributes
    Optional fields
```

Each of these will be discussed separately.

Image data and codec

The *Image data and codec* parts of an NTNDArray are composed of the following fields:

```
value_t value // Image data
codec_t codec // Codec
```

where:

value

An array which encodes an N-dimensional array containing the data for the image itself.

codec

Information on the how the data in value encodes the N-dimensional array.

A *value_t* is implemented as a pvData Field of type "union" with the following form:

```
value_t:=

union
    boolean[] booleanValue
    byte[]    byteValue
    short[]   shortValue
    int[]     intValue
    long[]    longValue
    ubyte[]   ubyteValue
    ushort[]  ushortValue
    uint[]    uintValue
    ulong[]   ulongValue
    float[]   floatValue
    double[]  doubleValue
```


A *codec_t* is implemented as a pvData Field of type "structure" of type ID "codec_t" with the following form:

```
codec_t :=

structure
    string name
    any    parameters
```

where:

name

The encoding scheme, e.g. the codec in the case of compressed data.

parameters

Any additional information required to interpret the data.

The `value` field stores a scalar array of one of the scalar types permitted by the definition of `value` above whose value MUST represent an N-dimensional scalar array of one of the permitted scalar types whose dimensions are given by the `dimension` field (see below). Note that the scalar type of the array stored in `value` MAY be different from that of the array it represents.

The `codec` field is a structure which describes how the N-dimensional scalar array is represented by the value of the scalar array stored in the `value` field.

The `name` field of the `codec` field (`codec.name`) is a string which identifies the scheme by which the data in `value` is encoded, such as an algorithm used to compress the data. If it is not the empty string, the value of the `codec.name` field SHOULD be namespace qualified.

The `parameters` field of the `codec` field (`codec.parameters`) is a field which contains any additional information required to interpret the data in `value`. The format and meaning of `codec.parameters` is `codec.name`-dependent.

When the value of the `codec.name` field is the empty string the data in `value` MUST represent an N-dimensional array of the same scalar type as the scalar array stored in `value` whose dimensions are given by the `dimension` field. The elements of the array stored in `value` MUST be the elements of the N-dimensional array laid out in row major order. In this case the length of the `value` array SHOULD equal the product of the dimensions and MUST be greater than or equal to it.

When the `codec.name` field value is not the empty string the interpretation of the data in the `value` field is dependent on the `codec` field. Any requirements on the type or length of the array stored in the `value` field are `codec`-dependent.

Any endianness information associated with a compression algorithm or other encoding SHOULD be encoded via the `codec` field, either through the `codec.name` or `codec.parameters` fields.

Similarly any information required to determine the scalar type of the N-dimensional array when the value of `codec.name` field is non-empty SHOULD also be encoded in the `codec` field.

Except for the above requirements, the meaning of the `codec` field, beyond the case of the empty `codec.name` string, is not currently specified.

Data sizes

The *Data sizes]* part of an NTNDArray is composed of the following fields:

```
long compressedSize
long uncompressedSize
```

where:

compressedSize

The size of the data in bytes after any compression or other encoding.

uncompressedSize

The size of the data in bytes before any compression or other encoding.

The value of the `compressedSize` field MUST be equal to the product of the length of the scalar array field stored in the `value` field and the size of the scalar type in bytes (i.e. 1, 2, 4 or 8 for signed or unsigned byte, short, int or long respectively, 1 for boolean, 4 for float and 8 for double).

The value of the `uncompressedSize` field MUST be equal to the product of the value of the `size` field of each element in the structure array `dimension` field (described below) and the size in bytes of the scalar type of the scalar array represented by `value`. If the number of elements of the `dimension` field is 0 the value of the `uncompressedSize` MUST be 0.

Dimensions

The *Dimensions* part of an NTNDArray is composed of the `dimension` field

```
dimension_t[] dimension
```

A *dimension_t* is implemented as a pvData Field of type "structure" of type ID "dimension_t" with the following form:

```
dimension_t :=

structure
    int     size
    int     offset
    int     fullSize
    int     binning
    boolean reverse
```

where:

size

The number of elements in this dimension of the array.

offset

The offset in this dimension relative to the origin of the original data source.

fullSize

The number of elements in this dimension of the the original data source.

binning

The binning (pixel summation, 1=no binning) in this dimension relative to original data source source.

reverse

The orientation (false=normal, true=reversed) in this dimension relative to the original data source source.

The number of elements in the value of the `dimension` field MAY be 0. A client SHOULD check for this case and take appropriate action.

If an NTNDArray represents a subregion of a larger region of interest of an original image, its `offset`, `binning` and `reverse` field values SHOULD be relative to the original image and its `fullSize` field value SHOULD be the size of the original.

dimension_t is analogous to NDDimension_t in areaDetector.

Unique ID and data timestamp

The *Unique ID and data timestamp* parts of an NTNDArray are composed of the following fields:

```
int     uniqueId
time_t  dataTimeStamp
```

where:

uniqueId

A number that SHOULD be unique for all NTNDArrays produced by a source after it has started.

dataTimeStamp

Timestamp of the data.

The value of `dataTimeStamp` MAY be different from that of the (optional) `timeStamp` field below.

The `uniqueId` and `dataTimeStamp` fields of NTNDArray correspond to the uniqueId and timeStamp fields respectively of an NDArray.

NTNDArray attributes

The *Attributes* part of an NTNDArray is composed of the field:

```
NTAttribute[] attribute
```

where *NTAttribute* is as defined by this standard, but is extended in this case as follows:

```
NTAttribute :=

structure
    string    name
    any       value
    string[]  tags          : opt
    string    descriptor
    alarm_t   alarm         : opt
    time_t    timeStamp     : opt
    int       sourceType
    string    source
```

where:

sourceType

The origin of the attribute

```
NDAttrSourceDriver   = 0,   /** Attribute is obtained directly from driver */
NDAttrSourceParam    = 1,   /** Attribute is obtained from an asyn parameter library */
NDAttrSourceEPICSPV  = 2,   /** Attribute is obtained from an EPICS PV */
NDAttrSourceFunct    = 3    /** Attribute is obtained from a user-specified function  */
```

source

The source string of this attribute.

Note that the optional descriptor field of *NTAttribute* is mandatory for attributes of an NTNDArray.

NTAttribute here is extended by the addition of the `sourceType` and `source` fields. `source` is a string which gives the origin of the attribute according to the value of the integer `sourceType` field as follows:

- For a `sourceType` of value `NDAttrSourceDriver` the `source` string SHOULD be the empty string.
- For a `sourceType` of value `NDAttrSourceParam` the `source` string SHOULD be the name of the asyn parameter from which the attribute value was obtained.
- For a `sourceType` of value `NDAttrSourceEPICSPV` the `source` string SHOULD be the name of the EPICS PV from which the attribute value was obtained.
- For a `sourceType` of value `NDAttrSourceFunct` the `source` string SHOULD be the name of the user function from which the attribute value was obtained.

The extension of *NTAttribute* is analogous to NDAttribute in areaDetector. The `name`, `descriptor`, `sourceType` and `source` fields correspond to the pName, pDescription, sourceType, pSource members of an NDAttribute respectively.

The attributes themselves are not defined by this standard.

For areaDetector applications the `attribute` field encodes the linked list of NDAttributes in an NDArray.

[Note: areaDetector currently defines two integer attributes, colorMode and bayerPattern, with descriptions “Color mode” and “Bayer pattern” respectively:

colorMode

> An attribute that describes how an N-d array is to be interpreted as an image, taking one of the values in this enumeration:

```
NDColorModeMono    = 0,    /** Monochromatic image */
NDColorModeBayer   = 1,    /** Bayer pattern image,
                               1 value per pixel but with color filter on detector */
NDColorModeRGB1    = 2,    /** RGB image with pixel color interleave,
                               data array is [3, NX, NY] */
NDColorModeRGB2    = 3,    /** RGB image with row color interleave,
                               data array is [NX, 3, NY]  */
NDColorModeRGB3    = 4,    /** RGB image with plane color interleave,
                               data array is [NX, NY, 3]  */
NDColorModeYUV444  = 5,    /** YUV image, 3 bytes encodes 1 RGB pixel */
NDColorModeYUV422  = 6,    /** YUV image, 4 bytes encodes 2 RGB pixel */
NDColorModeYUV411  = 7     /** YUV image, 6 bytes encodes 4 RGB pixels */
```

bayerPattern

An attribute valid when colorMode is NDColorModeBayer providing additional information required for the interpretation of an N-d array as an image in this case, taking one of the values in this enumeration:

```
NDBayerRGGB        = 0,    /** First line RGRG, second line GBGB... */
NDBayerGBRG        = 1,    /** First line GBGB, second line RGRG... */
NDBayerGRBG        = 2,    /** First line GRGR, second line BGBG... */
NDBayerBGGR        = 3     /** First line BGBG, second line GRGR... */
```

Other areaDetector attributes are user-defined.]

NTContinuum

NTContinuum is the EPICS V4 Normative Type used to express a sequence of point values in time or frequency domain. Each point has N values (N>=1) and an additional value which describes the index of the list. The additional value is carried in the `base` field. The `value` field carries the values which make up the point in index order.

An additional `units` field gives a units string for the N values and the additional value.

```
NTContinuum :=

structure
    double[]   base
    double[]   value
    string[]   units
    string     descriptor    :opt
    alarm_t    alarm         :opt
    time_t     timeStamp     :opt
```

The number of values in a point must be derived as:

Nvals = len(value)/len(base)

And the following invariant must be preserved:

len(units)-1 == Nvals

For points (A_i, B_i, C_i) for indices i = 1, 2, 3 the `value` array is:

[A_1, B_1, C_1, A_2, B_2, C_2, A_3, B_3, C_3]

NTHistogram

NTHistogram is the EPICS V4 Normative Type used to encode the data and representation of a (1 dimensional) histogram. Specifically, it encapsulates frequency binned data.

For 2d histograms (i.e. both x and y observations are binned) and n-tuple data (e.g. land masses of different listed countries) see NTMatrix or NTTable.

```
NTHistogram :=

structure
    double[]   ranges                    // The start and end points of each bin
    (short[] | int[] | long[])  value    // The frequency count, or otherwise value, of each bin
    string     descriptor    :opt
    alarm_t    alarm         :opt
    time_t     timeStamp     :opt
```

Interpretation

One NTHistogram gives the information required to convey a histogram representation of some underlying observations. It does not convey the values of each of the observations themselves.

The number of bins is given by the length of the `value` array. `ranges` indicates the low value and high value of each bin. The range for *bin(i)* is given by *ranges(i)* to *ranges(i+1)*. Specifically, since we want end points of both the first bin and last bin included, all bin intervals except the last one, MUST be *right half open*; from that bin's low value *ranges(i)* (included) to that bin's high value *ranges(i+1)* (excluded). The last bin MUST be fully *open* (low and high value included).

A log plot histogram (in which the independent variable x is binned on a log scale), would be communicated using a range array of decades (1.0E01, 1.0E02, 1.0E03 etc).

Validation

The array length of `ranges` MUST be the array length of `value` + 1.

NTAggregate

NTAggregate is the EPICS V4 Normative Type to compactly convey data which combines several measurements or observation. NTAggregate gives simple summary statistic bib:agg about the central tendency and dispersion of a set of data points.

Use cases: for instance, an NTAggregate could be used to summarize the value of one beam position offset reading over some number of pulses (N). It also includes the time range of the sampled points, so it could be used for time domain rebasing. For instance, an FPGA sending data at 10KHz, and you want to display its output, but you don't want to display at the native rate. Also, it could be used for transmitting or storing compressed archive data.

NTAggregate doesn't cover the shape of a distribution so it only reasonably helps you do symmetrical distributions (no skewness or kurtosis), and it doesn't include any help for indicating the extent of dependency on another variable (correlation).

```
NTAggregate :=

structure
    double     value                // The center point of the observations,
                                    // nominally the mean.
    long       N                    // Number of observations
    double     dispersion      :opt  // Dispersion of observations;
                                      // nominally the Standard Deviation or RMS
    double     first           :opt  // Initial observation value
    time_t     firstTimeStamp  :opt  // Time of initial observation
    double     last            :opt  // Final observation value
    time_t     lastTimeStamp   :opt  // Time of final observation
    double     max             :opt  // Highest value in the N observations
    double     min             :opt  // Lowest value in the N observations
    string     descriptor      :opt
    alarm_t    alarm           :opt
    time_t     timeStamp       :opt
```

where:

value

The summary statistic of the set of observations conveyed by this NTAggregate. For instance their arithmetic mean.

N

The number of observations summarized by this NTAggregate.

dispersion

The extent to which the observations are centered around the `value`. For instance, if the `value` contains a mean, then the dispersion may be the variance or the standard deviation. The `descriptor` should indicate which.

first

The value of the temporally first observation conveyed by this NTAggregate.

firstTimeStamp

The time of observation of the temporally first observation conveyed by this NTAggregate.

last

The value of the temporally final observation conveyed by this NTAggregate.

lastTimeStamp

The time of observation of the temporally final observation conveyed by this NTAggregate.

max

The numerically largest value in the set of observations conveyed by this NTAggregate.

min

The numerically smallest value in the set of observations conveyed by this NTAggregate.

Interpretation

One NTAggregate instance describes some number (given by N) of observations. If firstTimeStamp and lastTimeStamp are given, then the N observations MUST have been taken over the period of time specified. If first, last, max or min are given, they MUST refer to the actual values of the N observations being summarized.

The `value` field value computed by server agents may be the arithmetic mean of the observation data being summarized by this NTAggregate, but NTAggregate does not normatively define that. Other measures of mean (geometric, harmonic) may be assigned. Indeed other measures of central tendency may be used. The interpretation to give an instance of an NTAggregate SHOULD be conveyed in the `descriptor`.

Where dispersion is a measure of the standard deviation, which estimator of the standard deviation [1/N or 1/(N-1)] was used, is also not defined normatively.

Appendix A: Possible Future Additions to this Specification

NTUnion

NTUnion would be a Normative Type for interoperation of essentially any data structure, plus description, alarm and timestamp fields.

```
NTUnion :=

structure
    anyunion_t   value
    string       descriptor       :opt
    alarm_t      alarm            :opt
    time_t       timeStamp        :opt
```

NTScalarMultiChannel

NTScalarMultiChannel is an EPICS V4 Normative Type that aggregates an array of values from different EPICS Process Variable (PV) channel sources of the same scalar type into a single variable.

Use cases: In a particle accelerator, a single NTScalarMultiChannel might include the data of a number of Beam Position Monitors' X offset values, or of a number of quadrupoles' desired field values.

```
NTScalarMultiChannel :=

structure
    scalar_t[]    value              // The channel values
    string[]      channelName        // The channel names
    string        descriptor         :opt
    alarm_t       alarm              :opt
    time_t        timeStamp          :opt
    int[]         severity           :opt
    int[]         status             :opt
    string[]      message            :opt
    long[]        secondsPastEpoch   :opt
    int[]         nanoseconds        :opt
    int[]         userTag            :opt
```

where:

value

The value from each channel.

channelName

The name of each channel.

alarm

The alarm associated with the NTScalarMultiChannel itself. `severity`, `status`, and `message` show the alarm for each channel.

timeStamp

The timestamp associated with the NTScalarMultiChannel itself. `secondsPastEpoch`, `nanoseconds` and `userTag` show the timestamp for each channel.

severity

The alarm severity associated with each channel.

status

The alarm status associated with each channel.

message

The alarm message associated with each channel.

secondsPastEpoch

The `secondsPastEpoch` field of the timestamp associated with each channel.

nanoseconds

The `nanoseconds` field of the timestamp associated with each channel.

userTag

The `userTag` field of the timestamp associated with each channel.

Appendix B: Normative Type Identifiers

This Appendix describes the Normative Type Identifiers of the abstract data types defined by this document. These are the strings which identify the type carried by a structure. In the pvAccess binding (which is at present the only one implemented for EPICS V4), the type ID of the structure MUST carry one of these identifier strings. In doing so, the structure instance declares itself to conform to the corresponding definition carried in this specification document.

The syntax of the Normative Type identifier is:

```
namespacename/typename:versionnumber
```

The Normative Type Identifier “Namespace Name” part, is:

```
epics:nt
```

The Normative Type Identifier “Type Name” and version number parts corresponding to this draft of the Normative Types Document (this document), MUST be valued as following:

Table 62 Type Names that may be used in the Type Name part of a Normative Type Identifier of an EPICS V4 Normative Type in the namespace of this draft of the Normative Types specification

Type Name	Version	Depends on	Short Description
NTScalar	1.0	(none)	A single scalar value.
NTScalarArray	1.0	(none)	An array of scalar values of some single type.
NTEnum	1.0	(none)	An enumeration list and a value of that enumeration.
NTMatrix	1.0	(none)	A real number matrix.
NTURI	1.0	(none)	A structure for encapsulating a Uniform Resource Identifier (URI).
NTNameValue	1.0	(none)	An array of scalar values where each element is named.
NTTable	1.0	(none)	A table of scalars, where each column may be of different scalar array type
NTAttribute	1.0	(none)	A key-value pair, with optional string tags, where the value is of any type.
NTMultiChannel	1.0	(none)	An array of PV names, their values, and metadata.
NTNDArray	1.0	NTAttribute 1.0	A pixel and metadata type, designed to encode a frame of data from detectors and cameras.
NTContinuum	1.0	(none)	Expresses a sequence of data points in time or frequency domain.
NTHistogram	1.0	(none)	An array of real number intervals, and their frequency counts. Expresses a 1D histogram.

Type Name	Version	Depends on	Short Description
NTAggregate	1.0	(none)	A mean value, standard deviation, and other metadata. Expresses the central tendency and dispersion of a set of data points.

For example, the type ID of a structure describing an NTScalar, must be valued “epics:nt/NTScalar:1.0”. The type ID of a structure describing an NTNDArray, must be valued “epics:nt/NTNDArray:1.0”.

Following drafts of this document MAY well correspond to the same Namespace Name and Type Names as used in this draft. Also note that the same namespace may well be used for a different collection of types or Type Names, as this document matures.

Bibliography

[bib:pvdata]

EPICS V4 Documentation page, Programmers’ Reference Documentation section (pvData).

[bib:pvaccess]

V4 Documentation page, Programmers’ Reference Documentation section (pvAccess).

[bib:epicsrecref]

EPICS Reference Manual, Philip Stanley, Janet Anderson, Marty Kraimer, APS, https://wiki-ext.aps.anl.gov/epics/index.php/RRM_3-14.

[bib:epicsappdev]

EPICS Input / Output Controller (IOC) Application Developer’s Guide Marty Kraimer, APS, 1994, http://www.aps.anl.gov/epics/base/R3-14/12-docs/AppDevGuide/.

bib:agg

Aggregate data, Wikipedia article, http://en.wikipedia.org/wiki/Aggregate_data.

bib:rdbservice

rdbService, example EPICS V4 service, https://github.com/epics-base/exampleJava/tree/master/src/services/rdbService.

bib:uri

Uniform Resource Identifiers (URI): Generic Syntax, http://www.ietf.org/rfc/rfc2396.txt.